Closures & iterators

Язык программирования Rust. Лекция 5 (часть 1).

Closures

Closures: syntax

```
fn add_one(x: i32) -> i32 { x + 1 }      // function

let add_one = |x: i32| -> i32 { x + 1 }; // closure, full form
let add_one = |x| x + 1;                 // types are inferred

let n = 10;
let add_n = |x| x + n;                   // captures `n` from the environment

assert_eq!(add_n(1), 11);
```

- Замыкание — анонимная функция, которая может захватывать переменные из окружения.
- Типы аргументов и возвращаемого значения выводятся (но фиксируются при первом использовании).

Capturing the environment

Компилятор выбирает самый "слабый" способ захвата, которого достаточно:

```
let s = String::from("hi");
let print = || println!("{s}");  // &s       — shared borrow

let mut count = 0;
let mut inc = || count += 1;     // &mut count — unique borrow
inc();

let t = String::from("bye");
let consume = || drop(t);        // t          — by value (move)
```

Пока замыкание `inc` живо, `count` нельзя читать снаружи — обычные правила borrow checker'a.

`move` closures

```
fn long_words<'a>(text: &'a str, min: usize) -> impl Iterator<Item = &'a str> {
    text.split_whitespace().filter(|w| w.len() >= min)
}
```

```
error[E0373]: closure may outlive the current function, but it borrows `min`,
              which is owned by the current function
help: to force the closure to take ownership of `min` (and any other referenced
      variables), use the `move` keyword
  |     text.split_whitespace().filter(move |w| w.len() >= min)
```

- `move` заставляет захватывать **всё** по значению.
- Нужен, когда замыкание переживает текущую функцию: возврат из функции, `thread::spawn`.

What is a closure?

Компилятор генерирует анонимную структуру с захваченными переменными:

```
let n = 10;
let add_n = |x: i32| x + n;

// roughly desugars to:
struct AddN<'a> {
    n: &'a i32,
}

impl Fn<(i32,)> for AddN<'_> {   // not stable syntax, just the idea
    fn call(&self, (x,): (i32,)) -> i32 { x + *self.n }
}
```

- У каждого замыкания свой уникальный тип, назвать его нельзя.
- Размер замыкания = размер захваченного (`size_of_val(&add_n) ==`

`Fn`, `FnMut`, `FnOnce` (1)

```
trait FnOnce<Args> {
    type Output;
    fn call_once(self, args: Args) -> Self::Output;
}

trait FnMut<Args>: FnOnce<Args> {
    fn call_mut(&mut self, args: Args) -> Self::Output;
}

trait Fn<Args>: FnMut<Args> {
    fn call(&self, args: Args) -> Self::Output;
}
```

`Fn`, `FnMut`, `FnOnce` (2)

Трейт	Receiver	Что может делать с захваченным	Сколько раз вызвать
`FnOnce`	`self`	переместить наружу	один
`FnMut`	`&mut self`	изменять	много, не параллельно
`Fn`	`&self`	только читать	много

Which trait does a closure implement?

Определяется телом замыкания, а не тем, как захвачены переменные:

```
let s = String::from("hi");
let print = move || println!("{s}");   // Fn + FnMut + FnOnce (despite `move`)

let mut count = 0;
let mut inc = || count += 1;           // FnMut + FnOnce

let t = String::from("bye");
let consume = || drop(t);              // FnOnce only
```

- Только читает захваченное → `Fn`
- Изменяет захваченное → `FnMut`
- Перемещает захваченное из себя → `FnOnce`

Errors

```
let s = String::from("hi");
let f = move || drop(s);
f(); f();
```

```
error[E0382]: use of moved value: `f`
note: closure cannot be invoked more than once because it moves the variable `s`
      out of its environment
```

```
fn call_twice<F: Fn()>(f: F) { f(); f(); }

let mut c = 0;
call_twice(|| c += 1);
```

```
error[E0594]: cannot assign to `c`, as it is a captured variable in a `Fn` closure
  | fn call_twice<F: Fn()>(f: F) { f(); f(); }
  |                              - change this to accept `FnMut` instead of `Fn`
```

Choosing a bound

```
impl<T> Option<T> {
    fn map<U, F: FnOnce(T) -> U>(self, f: F) -> Option<U> { ... }  // called at most once
}

trait Iterator {
    fn map<B, F: FnMut(Self::Item) -> B>(self, f: F) -> Map<Self, F> { ... } // many times
}

fn spawn<F: FnOnce() -> T + Send + 'static, T>(f: F) -> JoinHandle<T> { ... }
```

Для автора функции: чем слабее требование, тем больше замыканий подойдёт.

- `FnOnce` — если вызываете не больше одного раза.
- `FnMut` — если вызываете много раз последовательно.
- `Fn` — если нужен вызов через `&self` (например, из нескольких потоков одновременно).

Passing and returning closures

```
fn apply<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(x) }  // static dispatch
fn apply2(f: impl Fn(i32) -> i32, x: i32) -> i32 { f(x) }  // same thing
fn apply_dyn(f: &dyn Fn(i32) -> i32, x: i32) -> i32 { f(x) } // dynamic dispatch

fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}

fn make_op(add: bool) -> Box<dyn Fn(i32) -> i32> {
    if add { Box::new(|x| x + 1) } else { Box::new(|x| x - 1) } // two different types
}

let ops: Vec<Box<dyn Fn(i32) -> i32>> = vec![Box::new(|x| x * 2), make_op(true)];
```

`fn` pointers

`fn` items and `fn` pointers

```rust
fn add_one(x: i32) -> i32 { x + 1 }

let item = add_one;                  // fn item: unique zero-sized type `fn(i32) -> i32 {add_one}`
let ptr: fn(i32) -> i32 = add_one;   // fn pointer: ordinary type, just an address

assert_eq!(size_of_val(&item), 0);
assert_eq!(size_of_val(&ptr), 8);
```

- `fn(i32) -> i32` — **тип** (указатель на код), а не трейт.
- Все функции с одной сигнатурой приводятся к одному типу `fn`-указателя.
- `fn`-указатели реализуют `Fn`, `FnMut`, `FnOnce` → их можно передавать везде, где ждут `impl Fn`.

Closures → `fn` pointers

```
fn apply_fn_ptr(f: fn(i32) -> i32, x: i32) -> i32 { f(x) }

apply_fn_ptr(add_one, 1);    // ok
apply_fn_ptr(|x| x * 2, 1);  // ok: closure without captures coerces to fn

let n = 10;
apply_fn_ptr(|x| x + n, 1);  // error
```

```
error[E0308]: mismatched types
  = note: expected fn pointer `fn(i32) -> i32`
                found closure `{closure@src/main.rs:5:18: 5:21}`
note: closures can only be coerced to `fn` types if they do not capture any variables
```

`fn` vs `Fn`

	`fn(A) -> B`	`impl Fn(A) -> B` / `F: Fn`	`dyn Fn(A) -> B`
Что это	тип (указатель)	трейт, статический вызов	trait object
Захват окружения	нет	да	да
Размер	1 указатель	размер захваченного	DST, нужен `&` / `Box`
Вызов	косвенный	прямой, инлайнится	через vtable

Когда `fn`: FFI с C, таблицы функций (`[fn(); N]`), когда точно не нужно состояние.

Iterators

`Iterator` trait

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;

    // + ~75 provided methods: map, filter, zip, fold, collect, ...
}
```

```
struct Fib { curr: u64, next: u64 }

impl Iterator for Fib {
    type Item = u64;
    fn next(&mut self) -> Option<u64> {
        let res = self.curr;
        self.curr = self.next;
        self.next = res.checked_add(self.next)?;  // stop on overflow
        Some(res)
    }
}
```

`IntoIterator` and `for` loops (1)

```
trait IntoIterator {
    type Item;
    type IntoIter: Iterator<Item = Self::Item>;
    fn into_iter(self) -> Self::IntoIter;
}

for x in collection { body }
// desugars to:
let mut iter = IntoIterator::into_iter(collection);
while let Some(x) = iter.next() { body }
```

`IntoIterator` and `for` loops (2)

Выражение	Тип элемента	Коллекция после цикла
`for x in v` / `v.into_iter()`	`T`	перемещена
`for x in &v` / `v.iter()`	`&T`	доступна
`for x in &mut v` / `v.iter_mut()`	`&mut T`	доступна

Adapters

Адаптеры принимают итератор и возвращают новый итератор:

```
let v: Vec<_> = (1..=10)
    .filter(|x| x % 2 == 0)          // 2, 4, 6, 8, 10
    .map(|x| x * x)                  // 4, 16, 36, 64, 100
    .skip(1)                         // 16, 36, 64, 100
    .take_while(|&x| x < 70)         // 16, 36, 64
    .enumerate()                     // (0, 16), (1, 36), (2, 64)
    .collect();
```

Ещё полезные: `zip`, `chain`, `flat_map`, `flatten`, `rev`, `step_by`, `peekable`, `fuse`, `inspect`, `scan`, `take`, `skip_while`, `filter_map`, `cloned` / `copied`.

Iterators are lazy

```
let v = vec![1, 2, 3];
v.iter().map(|x| x * 2);
```

```
warning: unused `Map` that must be used
  = note: iterators are lazy and do nothing unless consumed
```

```
let it = v.iter().map(|x| { println!("map {x}"); x * 2 });
println!("before collect");
let doubled: Vec<_> = it.collect();
// before collect
// map 1
// map 2
// map 3
```

How adapters work

```
pub struct Map<I, F> {
    iter: I,
    f: F,
}

impl<B, I: Iterator, F: FnMut(I::Item) -> B> Iterator for Map<I, F> {
    type Item = B;

    fn next(&mut self) -> Option<B> {
        self.iter.next().map(&mut self.f)
    }
}
```

- Адаптер — обычная структура, хранящая исходный итератор и замыкание.
- `FnMut`, потому что `f` вызывается на каждом `next`.
- Всё мономорфизируется и инлайнится → zero-cost: цепочка

Consumers

```
let v = vec![1, 2, 3];

v.iter().sum::<i32>()                    // 6
v.iter().fold(0, |acc, x| acc + x)       // 6
v.iter().count()                         // 3
v.iter().max()                           // Some(3)
v.iter().any(|&x| x > 2)                 // true
v.iter().all(|&x| x > 0)                 // true
v.iter().find(|&&x| x == 2)              // Some(2)
v.iter().position(|&x| x == 2)           // Some(1)
v.iter().for_each(|x| println!("{x}"));
```

Короткозамыкающие (`any`, `all`, `find`, `position`) останавливаются, как только ответ известен.

`collect`

```
fn collect<B: FromIterator<Self::Item>>(self) -> B;
```

```
let names: Vec<String> = ["a", "b"].into_iter().map(String::from).collect();
let s: String = ['a', 'b'].into_iter().collect();
let m: HashMap<&str, i32> = [("a", 1), ("b", 2)].into_iter().collect();
let set = v.iter().collect::<HashSet<_>>();  // turbofish
```

Можно собрать в `Result` / `Option` — остановится на первой ошибке:

```
let res: Result<Vec<i32>, _> = ["1", "2", "x"].iter().map(|s| s.parse::<i32>()).collect();
// Err(ParseIntError { kind: InvalidDigit })
```

Constructing iterators

```
use std::iter;

let mut state = 0;
let it = iter::from_fn(move || {                    // closure returns Option<Item>
    state += 1;
    (state <= 3).then_some(state)
});                                                 // 1, 2, 3

let powers = iter::successors(Some(1u32), |x| x.checked_mul(2)); // 1, 2, 4, ..., 2^31

let mut c = 0;
let evens = iter::repeat_with(move || { c += 2; c }).take(3);    // 2, 4, 6

let ones = iter::repeat(1).take(3);
let nothing = iter::empty::<i32>();
let single = iter::once(42);
```

Returning iterators

```rust
fn long_words(text: &str, min: usize) -> impl Iterator<Item = &str> {
    text.split_whitespace().filter(move |w| w.len() >= min)
}

fn evens(v: &[i32]) -> Box<dyn Iterator<Item = &i32> + '_> {
    Box::new(v.iter().filter(|x| *x % 2 == 0))
}
```

- `impl Iterator` — точный тип скрыт (там `Filter<SplitWhitespace, {closure}>`), без аллокаций.
- `Box<dyn Iterator>` — если нужно вернуть разные типы итераторов из разных веток.
- `move` нужен, чтобы замыкание забрало `min`, а не ссылалось на локальную переменную.

Useful tricks

```
let mut it = v.iter();
let first_two: Vec<_> = it.by_ref().take(2).collect(); // borrow, don't consume
let rest = it.next();                                  // Some(3)

let mut it = [1, 2, 3, 10, 4].into_iter().peekable();
while let Some(x) = it.next() {
    if let Some(&next) = it.peek() { /* look ahead without consuming */ }
}

let it = iter.fuse();  // guarantees None forever after the first None
```